LeRobot: от телоуправления к датасету и MuJoCo



Яндекс Образование

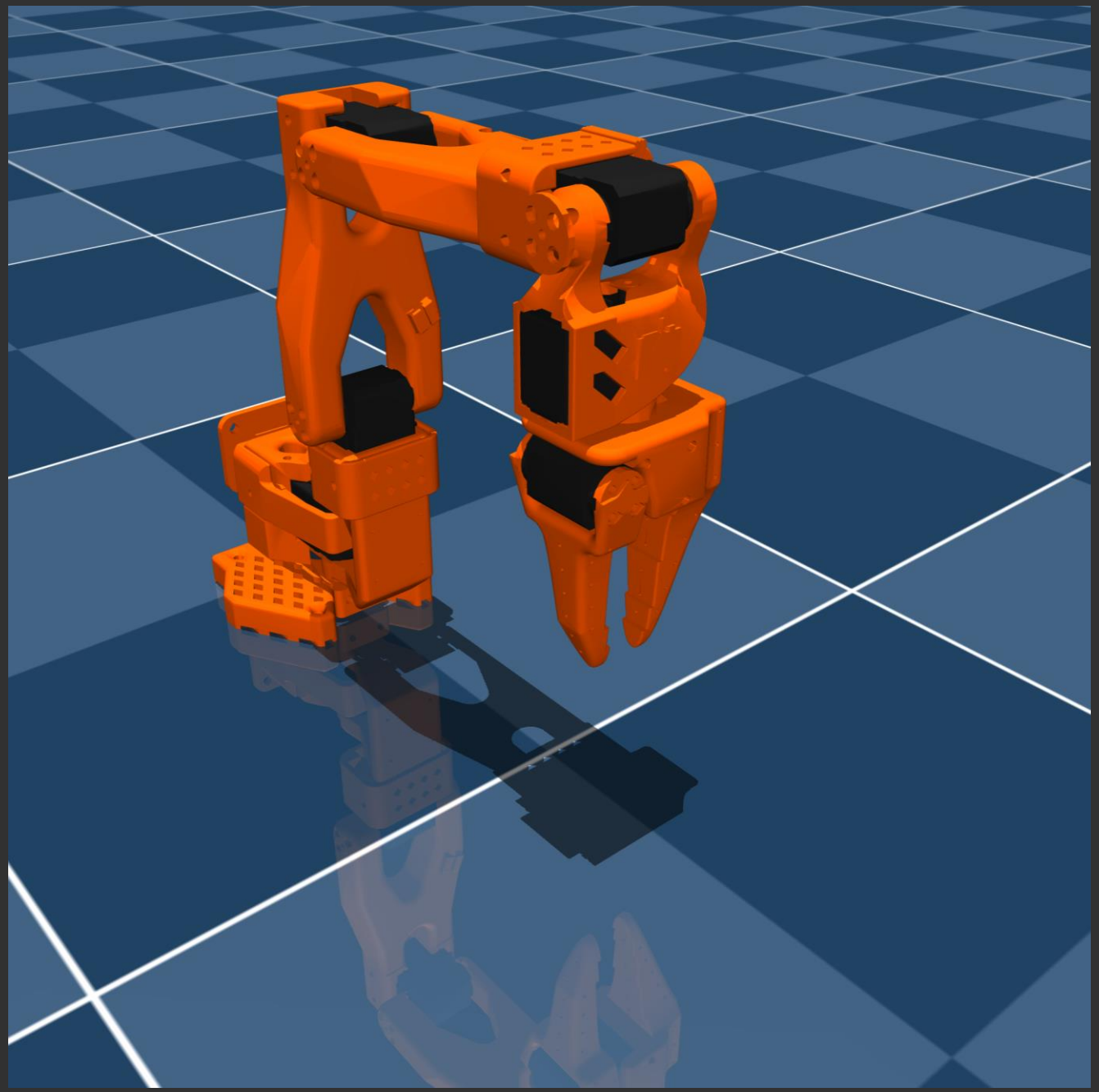
SO-101

Доступный робот-манипулятор для обучения робототехнике: на нём можно управлять рукой, собирать данные и обучать модели.



LeRobot

- Open-source экосистема Hugging Face для роботов: телоуправление, запись данных, обучение и запуск
- Единый workflow: calibrate → teleoperate → record → replay → train
- Формат LeRobotDataset v3: состояния и действия, видео с камер и метаданные в одной системе
- Можно работать через готовые CLI-команды и не писать большой пайплайн с нуля



Подготовка окружения: где запускать и что важно не сломать

01

В терминале, из корня проекта

```
./install.sh
source .venv/bin/activate

pip install -r requirements.txt
```

Очень важное правило

Не использовать `pip install lerobot` напрямую. В README специально сказано, что это может перезаписать зависимости других пакетов. Нужный путь — через `./install.sh`, где зависимости ставятся аккуратно.

Подключение робота

02

Шаг 1. Найти порт

`python -m lerobot.find_port`

Нужно, чтобы понять, на каком устройстве висит робот. Это первый чек: ноутбук вообще “видит” железо или нет.

Шаг 2. Калибровать

`python -m lerobot.calibrate`

Калибровка нужна, чтобы leader и follower корректно интерпретировали положение суставов. Без неё управление может быть рваным или просто неверным.

Шаг 3. Попробовать телеуправление

`python -m lerobot.teleoperate`

Один манипулятор ведёт, другой повторяет.

MUJOCO

Коротко

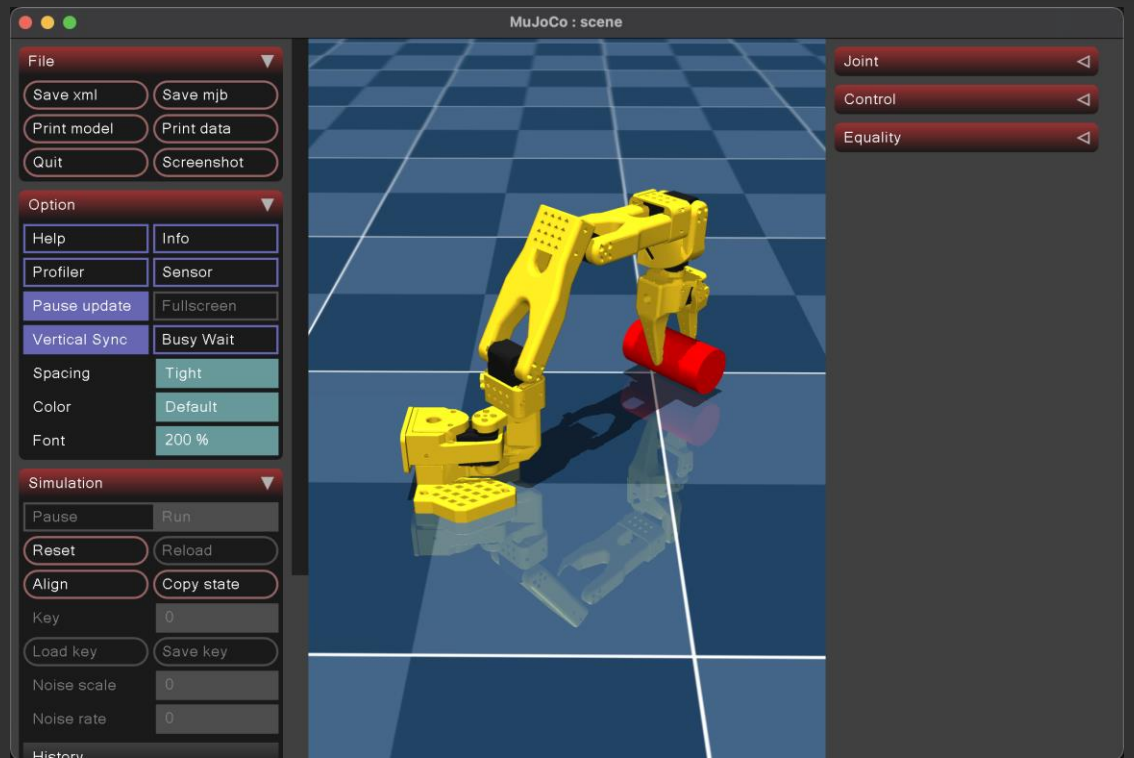
MuJoCo — это физический симулятор. Он позволяет запускать сцену с роботом и объектами так, чтобы движение, столкновения и состояние мира обновлялись по правилам физической модели.

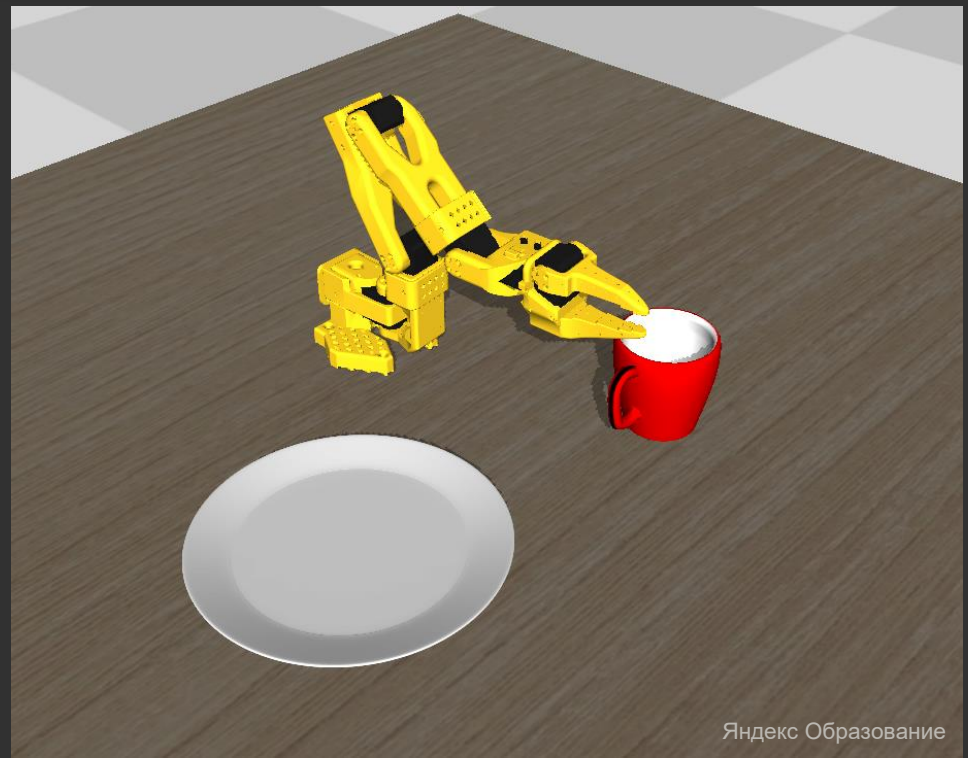
Для воркшопа

Он даёт безопасное место для тренировки, отладки и демонстраций. Можно быстро сбросить сцену, повторить попытку и показать ту же задачу много раз.

Для ML

Симулятор помогает масштабировать эксперименты: собирать больше данных, воспроизводить эпизоды и проверять, что именно делал робот на каждом шаге.





Сбор данных в симуляторе

03

Главная команда блока

```
python -m collect_data.run
```

Что делает этот запуск в общем виде

Скрипт берёт конфиг, поднимает среду MuJoCo, подключает источник управления, создаёт или загружает датасет и запускает цикл записи демонстраций. После завершения он корректно закрывает контроллер и viewer.

Где лежат настройки

В `collect_data/config.py`:

```
repo_name = "so101_pnp "
root = ./demo_data

num_demo = 3

use_master_arm = True

task_name = "Поставить кружку на
тарелку «

xml_path =
./asset/example_scene_y.xml

fps = 20
```

Пакет collect_data: из каких файлов он состоит

run.py

`main()` собирает всё вместе:
1) читает default_config()
2) создаёт env
3) подключает controller
4) создаёт/загружает dataset
5) запускает collect_demonstrations()
6) в finally закрывает ресурсы.

config.py

Описание `CollectDataConfig`.

Здесь сосредоточены параметры запуска: имя датасета, путь к данным, число демонстраций, режим master arm, порты, task_name, xml_path, fps, image_size и writer threads/processes.

controller.py

Логика подключения мастер-руки.

Если `use_master_arm=False`, возвращается `None` и печатается сообщение про клавиатуру. Если `True`, создаётся `SO101MasterArmController`, выполняется connect() и later disconnect().

env_runner.py

Самая "живая" часть.

Создаёт env, датасет и содержит основной цикл записи кадров и эпизодов.

Не "один огромный файл на всё", а маленькая инженерная система:

- один файл хранит настройки
- один отвечает за подключение устройства
- один — за сценарий запуска
- один — за саму механику записи

Даже небольшой робототехнический проект лучше делать модульно.

__init__.py

Небольшой служебный файл пакета: позволяет запускать модульную структуру аккуратно.

Главный цикл записи в collect_data/env_runner.py

1 env.step_env() + loop_every(fps)

Среда и viewer обновляются в заданном темпе.

2 Получаем action

Либо с мастер-руки, либо с клавиатуры через teleop_robot().

3 Проверяем reset / начало движения

Z сбрасывает сцену, движение запускает запись.

4 Снимаем наблюдения

Берём ee_pose и изображения с камер, ресайзим кадры.

5 Делаем шаг среды

env.step(action), получаем commanded_q.

6 Пишем frame в dataset

add_frame(): image, wrist_image, state, action, obj_init.

7 Сохраняем эпизод

X — ручное сохранение, success — автосохранение.

Что именно попадает в датасет и почему

observation.image

Кадр основной камеры: что “видит” агент снаружи.

observation.wrist_image

Кадр с камеры у запястья: полезен для близкого контакта с объектом.

observation.state

Состояние рабочего конца / поза, с которой модель соотносит действие.

action

Командуемые углы суставов, то есть “что делать дальше”.

obj_init

Стартовое положение объектов — нужно для корректного replay.

```
<= v2.1 (episode-based)
├── data/
│   └── chunk-000/
│       ├── episode_000000.parquet
│       ├── episode_000001.parquet
│       └── ...
└── videos/
    └── image_key/
        └── chunk-000/
            ├── episode-000.mp4
            ├── episode-001.mp4
            └── ...

v3.0 (file-based)
├── data/
│   └── chunk-000/
│       ├── file-000.parquet
│       ├── file-001.parquet
│       └── ...
└── videos/
    └── chunk-000/
        └── image_key/
            ├── file-000.mp4
            ├── file-001.mp4
            └── ...
```

Управление в MuJoCo с клавиатуры

Главная команда блока

```
python -m collect_data.run
```

Что происходит в режиме keyboard control

Если use_master_arm = False, источник действия — не мастер-рука, а клавиатура. Тогда скрипт использует delta_joint_angle: участник маленькими шагами изменяет углы суставов и открывает/закрывает захват.

Клавиши

Q / A — shoulder_pan ±
W / S — shoulder_lift ±
E / D — elbow_flex ±
I / K — wrist_flex ±
O / L — wrist_roll ±
Space — переключить gripper
Z — reset текущего эпизода

Идея

```
# смысл простой:
# нажал клавишу → изменил сустав →
# увидел результат сразу в сцене
```

Визуализация собранного эпизода

Как запускать

```
python -m visualize_data.run
python -m visualize_data.run --episode 0
python -m visualize_data.run --save-stats
```

Что делает этот модуль

Он загружает уже записанный датасет, выбирает один эпизод или все сразу, поднимает MuJoCo-среду и воспроизводит сохранённые действия. Одновременно можно увидеть и движение в сцене, и записанные изображения камер.

Почему это очень полезно

Replay быстро отвечает на важный вопрос: “мы правда записали то, что думали записать?”

Это мощный инструмент проверки качества данных.

Если указан --episode

Показывается конкретный эпизод. Это удобно для разбора одного удачного или одного неудачного примера.

Если указан --save-stats

После запуска дополнительно сохраняется `stats.json` — пригодится для анализа метаданных и статистики датасета.

Пакет visualize_data: основные файлы и роли

run.py

Разбирает аргументы командной строки, собирает `VisualizeDataConfig`, загружает датасет и выбирает: replay одного эпизода или последовательный replay всех эпизодов.

config.py

Здесь лежат параметры: repo_name, root, episode, xml_path, fps, save_stats.

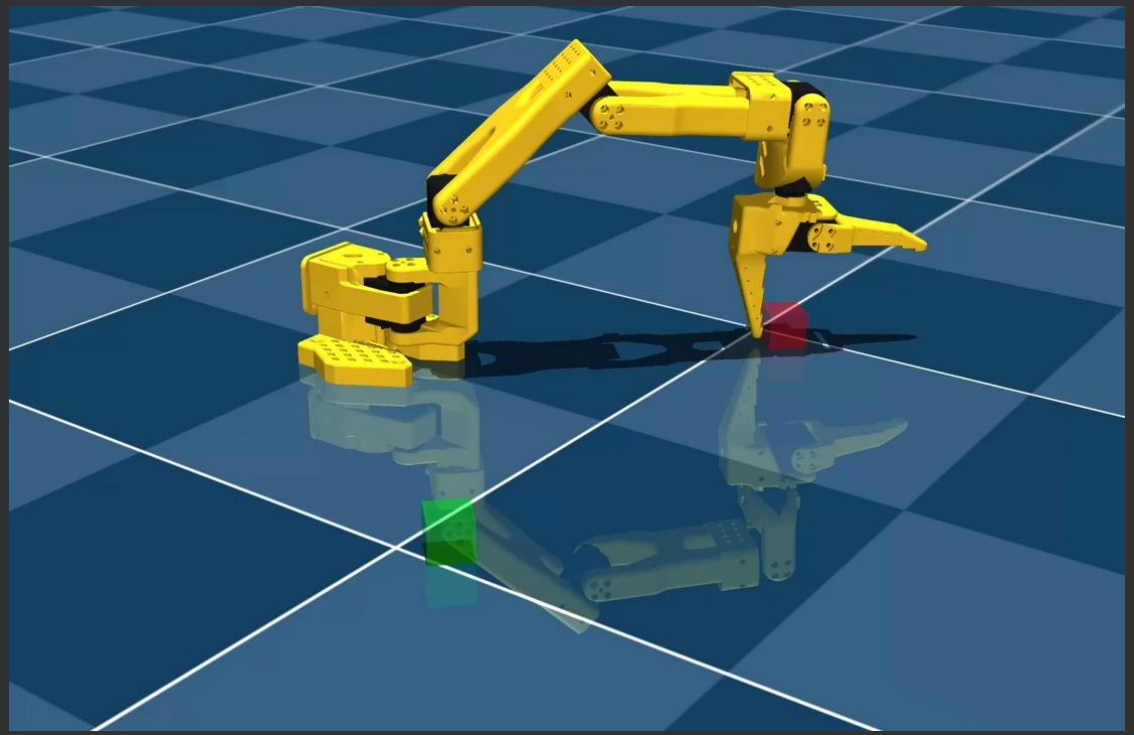
То есть всё, что определяет, какой датасет и в каком режиме проигрывать.

dataset.py

Содержит `EpisodeSampler`, загрузку LeRobotDataset, создание dataloader-ов для одного или всех эпизодов и сохранение stats.json.

env_runner.py

Поднимает среду, делает replay действия за действием, восстанавливает obj_init и прокидывает записанные camera frames в render.



__init__.py

Маркер пакета: помогает модульному запуску через `python -m visualize_data.run`.

Как устроен replay внутри visualize_data

1 load_dataset()

Загружается LeRobotDataset и печатаются базовые размеры: число эпизодов и кадров.

2 EpisodeSampler

Выбирает диапазон frame-ов конкретного эпизода по from/to индексам.

3 DataLoader

Кадры подаются по одному в нужном порядке.

4 env.reset()

Перед стартом воспроизведения MuJoCo сцена сбрасывается.

5 obj_init → set_obj_pose()

На первом кадре восстанавливается стартовая поза объектов.

6 action → env.step()

Сохранённое действие снова подаётся в среду.

7 camera frames → render()

Записанные картинки камер подмешиваются в отображение.

8 конец эпизода

Либо цикл повторяется, либо переключаемся на следующий episode.

В LeRobot можно запустить обучение одной командой

Идея запуска

```
lerobot-train \
  --dataset.repo_id=<ваш_датасет> \
  --policy.type=act \
  --output_dir=outputs/train/act_workshop \
  --job_name=act_workshop \
  --policy.device=cuda \
  --policy.push_to_hub=false
```

Что означает эта команда человеческим языком

“Возьми мой датасет, создай policy типа ACT, запусти обучение на GPU, складывай чекпоинты в указанную папку и не пытайся публиковать модель автоматически”.

Почему это выглядит магически

Потому что большой кусок ML-инженерии уже упакован в готовый инструмент.

Важно!

Одна команда не гарантирует хорошую модель. Нужны хорошие демонстрации, вычисления и понятная задача.

Почему хорошие демонстрации собирать не так просто

Качество движения

Если траектория дёрганая, случайная или противоречивая, то модель выучит именно это. Датасет запоминает не “правильный смысл”, а конкретные примеры поведения.

Повторяемость

Важно, чтобы похожие задачи решались похожим образом. Иначе модель получает конфликтующие сигналы и хуже обобщает поведение.

Стартовые условия

Положение объектов, ракурс, темп движения — всё это влияет на то, насколько полезным окажется эпизод для будущего обучения.